

Structured Output

让 Agent 的输出可校验、可恢复、可执行——从 JSON Schema 到业务协议的完整链路

学习目标

- 1 说明 Agent 为什么不能用自然语言直接驱动程序执行
- 2 使用 JSON Schema 定义字段、类型、必填项和枚举范围
- 3 使用 Pydantic 定义协议、生成 Schema、校验 JSON 并得到 Python 对象
- 4 把模型原生 Structured Output 与服务端校验配合使用
- 5 实现有次数上限的输出纠错，并区分格式错误与其他错误
- 6 连续失败后返回明确的降级或失败结果，保证非法输出不触发工具调用

Prompt 已规定格式，为何还需要 Structured Output?

Agent 开发工程师不能只写一段 Prompt 就指望模型输出正确——必须亲自完成四件工程工作：

工程工作	需要人决定什么
定义协议	允许哪些动作，每个动作需要哪些字段
实现校验	什么结果可以进入 Loop，什么结果必须拒绝
设计修复	哪些错误可以让模型重写，最多重写几次
处理失败	何时停止，何时降级，何时允许读取缓存

自然语言解析的局限与 JSON Schema

关键词解析：脆弱且不可扩展

用户问「公司差旅报销需要哪些材料」，最初实现可能使用关键词匹配：

parse_action — 关键词匹配

```
def parse_action(text: str) -> str:
    if "搜索" in text or \
        "search_docs" in text:
        return "search_docs"
    return "finish"
```



JSON Schema：统一声明输出结构

用统一格式声明字段名、类型、必填项、枚举值和额外字段规则：

AgentDecision — JSON Schema

```
{
  "type": "object",
  "properties": {
    "action": {
      "type": "string",
      "enum": ["search_docs", "finish"]
    },
    "query": { "type": ["string", "null"] },
    "answer": { "type": ["string", "null"] }
  },
  "required": ["action", "query", "answer"],
  "additionalProperties": false
}
```

Pydantic：从 Schema 到可执行协议

Pydantic 的核心价值，是让同一个 Model 同时承担四项职责：

职责	在 Agent 中的作用
定义类型	明确 Loop 最终接收什么对象
生成 Schema	把同一份结构提供给模型 API
校验输出	拦截字段、类型和业务组合错误
反序列化	把 JSON 转为可直接使用的 Python 对象

同一个 Model，四步闭环

字段规则

定义字段类型、必填项、枚举范围与跨字段约束

生成 Schema

model_json_schema() 自动导出 JSON Schema

校验 + 对象

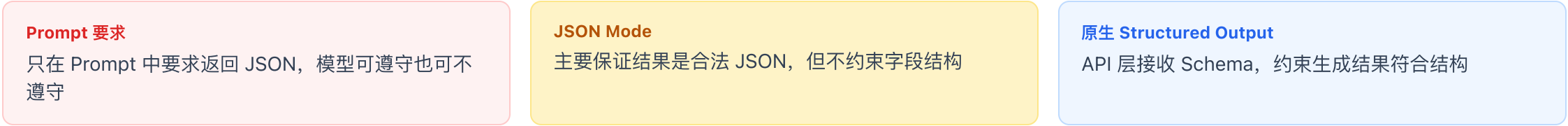
model_validate_json() 校验并返回 Python 对象

可诊断错误

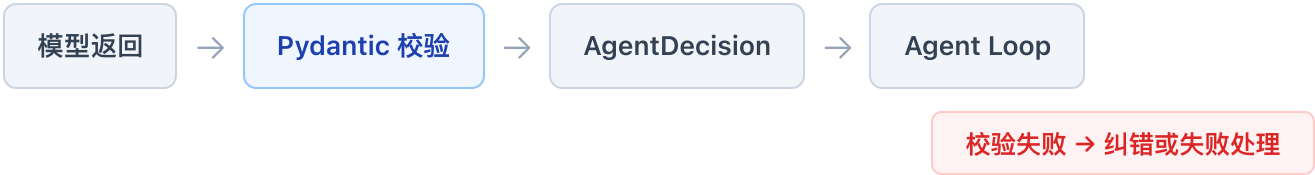
每条错误说明位置、类型和原因，非「解析失败」

模型原生 Structured Output 解决了什么？

三种输出约束的层级递进



无论输出来自哪种模型，都先经过同一个服务端边界



text_format = AgentDecision 让 SDK 根据 Pydantic Model 提供 Schema，并把正常结果解析成类型对象。但原生 Structured Output 仍需处理拒答、截断和业务错误——服务端校验不可省略。

三层校验与「全有或全无」原则

一次校验依次覆盖三层边界

校验层	典型问题	结果
JSON 语法	引号或括号没有闭合	无法解析
字段结构	字段缺失、类型错误、未知动作	无法构造对象
业务组合	finish 没有答案	业务校验拒绝

CORE PRINCIPLE

全有或全无

action 缺失不能默认 finish；混入未知字段也不能删除后继续。只有完整通过校验的 AgentDecision 才能离开这一层。

不同错误最后都不能进入 Loop，但保留失败层次后，系统才能选择正确的处理方式。结构正确 ≠ 内容正确——Structured Output 解决前者，让后者有稳定对象可继续检查。

把校验错误反喂给模型，并限制重试

第一次输出失败常常只是漏字段或类型错误。Pydantic 已给出准确原因，Harness 可把原因反馈给模型修正。

有效的纠错反馈

三个必答问题

哪里错了 · 为什么错 · 允许修改什么

Repair Loop 只重新生成决策，不执行任何工具。三次生成失败不会造成三次业务副作用。

哪些错误不应该反喂给模型？

失败类型	正确处理
字段缺失、类型错误、组合冲突	反馈校验错误，让模型修复
网络超时、限流	由模型调用层按退避策略重试
权限不足	立即拒绝或请求授权
知识库不可用	停止、切换服务或转人工

多次修复仍失败：降级与停止

OK

成功

模型给出合法决策，可进入 Agent Loop

DEGRADED

降级

模型失败，但系统找到满足条件的替代结果

FAILED

失败

没有任何结果可安全使用，decision = None

缓存什么时候才是合法的降级结果？

缓存不是「出错后随便找一条旧答案」。只允许回退到同一问题近期成功生成的只读答案，并检查：

任务键

协议版本

有效期

处理顺序：修复 → 缓存 → 失败



输出格式 = 稳定的业务协议

设计 Schema 前先回答五个业务问题

- 1 谁消费输出？
- 2 输出决定什么？
- 3 允许哪些动作？
- 4 每种动作需要哪些数据？
- 5 失败后怎样处理？

字段不需要追求复杂。action、query、answer 直接描述下游消费的数据，已经足够。未被程序使用的字段只会增加误用和维护成本。

CLASSROOM EXERCISE

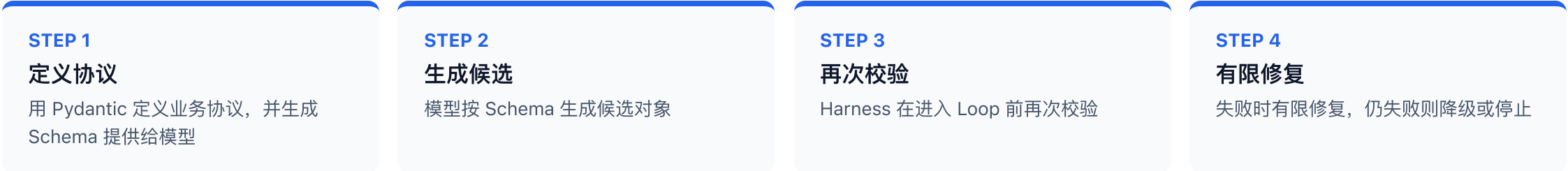
工单分类 Agent

定义结构化输出，只能执行两种动作：route_to_team（转交团队）和 ask_user（追问用户）。

- 1 用 Pydantic 定义 action、team 和 question
- 2 转交时必须有 team，追问时必须有 question
- 3 禁止额外字段
- 4 用 model_validate_json() 校验输出
- 5 校验错误反馈模型，最多修复两次
- 6 连续失败后返回 failed，不能猜测团队继续执行

本节小结

Prompt 负责告诉模型「应该怎样决策」；Structured Output 进一步判断「系统实际收到了什么决策」。完整链路收束为四步：



核心认知
真正需要 Agent 工程师完成的，不是让模型输出一段看起来像 JSON 的文本，而是确定动作和字段、写出业务规则、划定重试预算、设计失败结果，并验证非法输出永远不会产生工具副作用。

→ **下一节：LLM Gateway**
把 Prompt、Schema、Pydantic 校验、错误处理和模型调用封装成统一入口